DocXAI

AI-Powered Document & Test Automation Engine

CGI - GCA Hackathon 2026 | Team 19 - MFG | .NET 10 • Vertex AI (Gemini 2.0 Flash) • Google Cloud Platform

1. Problem Statement

Converting design documents into functional specifications and test cases is **manual, slow, and error-prone**. Business analysts write specs by hand; QA engineers separately author test scripts - often weeks apart. Key pain points:

- **Slow turnaround:** Specs take days to draft; test scripts take additional days, delaying the entire SDLC.
- **Inconsistency:** Manual translation introduces gaps, ambiguities, and missed requirements.
- **No traceability:** No automated link between design → spec → tests.
- **Version friction:** Iterations managed in shared drives or email files get overwritten or lost.
- **Repetitive effort:** Teams repeat the same read → write → test pattern with no reuse or automation.

2. Use Case

A project team receives a design document (`.docx` or Markdown). They need to:

1. Produce a comprehensive **Functional Specification** capturing all requirements and edge cases.
2. Generate an executable **Playwright test suite** (TypeScript) that validates the system against the spec.
3. Store all artifacts with **automatic versioning** - nothing is ever overwritten.

> **Key Insight:** The functional spec and test cases are two views of the same requirements. Generating both from a single document via AI ensures 100% traceability and eliminates the "telephone game" between design, spec, and QA.

3. Proposed Solution

DocXAI automates the design → spec → test pipeline using Gemini 2.0 Flash. Built as a clean-architecture .NET 10 app on Cloud Run with a browser UI for human-in-the-loop control.

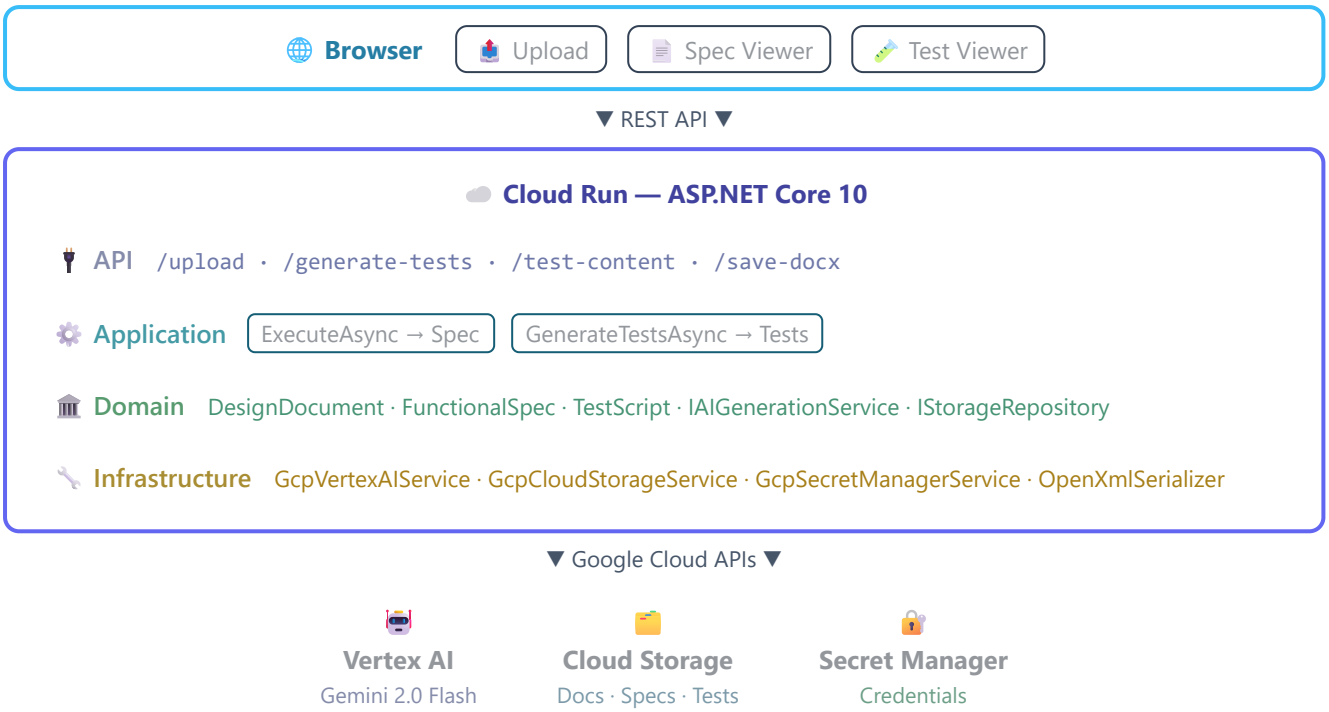
Core Workflow

1. **Upload** - drag a design doc into the browser, provide a Solution Name.
2. **AI Spec Generation** - Gemini produces a Functional Specification (Markdown).
3. **Review & Save** - spec displayed in a Markdown viewer; copy or download as `.docx`.
4. **On-Demand Test Gen** - user clicks "Generate Test Cases"; Gemini produces Playwright TypeScript.
5. **Review Tests** - syntax-highlighted viewer with download and copy.

Key Design Decisions

- **Human-in-the-loop:** test generation is decoupled - user reviews spec first.
- **Auto versioning:** `_v1`, `_v2`, `_v3` ... - never overwrites.
- **Organised storage:** `{Sol}/functional/` for specs, `{Sol}/test cases/` for tests.
- **Zero-framework frontend:** vanilla JS - no dependencies, instant load.

4. Solution Architecture



5. Data Flow

```
PHASE 1 — Spec Generation

 User uploads design.docx
     │  + solutionName
     ▼
POST /upload
     │
     ├─► Save to GCS: {Sol}/{file}
     ├─► Extract text (OpenXML)
     ├─► Vertex AI (Gemini)
     │   "Generate functional spec"
     ├─► Serialize to .docx
     ├─► Save to GCS:
     │   {Sol}/functional/
     │   {Sol}_functional_v1.docx
     └─► Return Markdown to browser
         → marked.js viewer
```

```
PHASE 2 — Test Generation (On-Demand)

 User clicks "Generate Test Cases"
     │  sends spec content + path
     ▼
POST /generate-tests
     │
     ├─► Vertex AI (Gemini)
     │   "Generate Playwright tests"
     │
     ├─► Save to GCS:
     │   {Sol}/test cases/
     │   {Sol}_testcases_v1.spec.ts
     │
     └─► Return TypeScript to browser
         → highlight.js viewer
```

6. Technology Stack

Component	Technology
Runtime	.NET 10, ASP.NET Core
Architecture	Clean / Onion (4 layers)
AI Model	Vertex AI — Gemini 2.0 Flash
Hosting	Google Cloud Run

7. Impact & Benefits

Metric	Before	After
Spec creation	2–5 days	< 2 min
Test creation	3–7 days	< 2 min
Traceability	Manual	100% auto
Versioning	Shared drives	Auto v1,v2...

Component	Technology
Storage	Google Cloud Storage
Secrets	Google Secret Manager
Test Framework	Playwright (TypeScript)
Doc Format	OpenXML (`.docx`)
Frontend	Vanilla JS, marked, hljs

8. Future Enhancements

- Test execution via Cloud Build / Azure Pipelines
- Jira / Azure DevOps user story creation
- Multi-model support (Gemini, Claude, GPT)
- Regression detection across spec versions
- Team auth, RBAC, shared workspaces